§ 5. Декомпозиция данных и запросов

В дереве отрезков иерархия делит диапазон на всё более мелкие части. У блочной структуры обычно только два масштаба: отдельные элементы и блоки примерно одинакового размера. Она выигрывает, когда сводку блока легко пересчитать, а более сложное дерево требует неудобного объединения или дорого поддерживает изменения.

Цель параграфа — научиться выводить размер блока из стоимости операций, различать разбиение по позициям и по значениям, ускорять прыжки и отделять тяжёлые объекты от лёгких. Нужны полуинтервалы, двоичный поиск, асимптотика и метод амортизации из § 1. Сначала рассмотрим статичное расположение элементов, затем перейдём к изменению самой последовательности.

5.1. Интерфейс и две границы запроса

Поддержим массив длины n, точечное присваивание `set(i,x)` и сумму `sum(l,r)` на $[l, r)$. Выберем целое $B \geq 1$. Блок b содержит позиции $[bB, \min(n, (b+1)B))$; последний блок может быть короче. Сохраним сам массив и сумму каждого блока. Инвариант: сводка блока равна сумме его текущих элементов.

Запрос состоит из неполного левого края, нескольких целых блоков и неполного правого края. Края читаем поэлементно, целые блоки — по сводке. Если обе границы лежат внутри одного блока, целых блоков нет; элементы нельзя учитывать дважды. Пустой запрос возвращает ноль.

Для $A = [2, 7, 1, 4, 3, 6, 5, 8, 9, 0, 2, 1]$, $B = 4$ суммы блоков равны 14, 22 и 12. Запрос $[2, 11)$ берёт элементы 1 и 4, целый блок с суммой 22 и элементы 9, 0, 2. Ответ равен 38. Присваивание `set(5,10)` меняет только второй блок: его сумма становится 26; тот же запрос теперь возвращает 42.

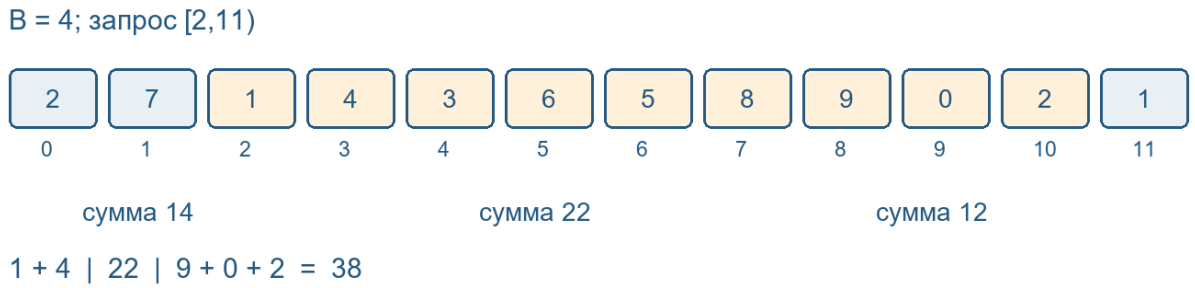


Рисунок 1: Два неполных края читаются поэлементно; середина запроса покрывается сводкой.

Стоимость запроса $O(B + n/B)$: на краях меньше $2B$ элементов, в середине не больше $\lceil n/B \rceil$ сводок. Построение и память линейны. Для суммы точечное присваивание обновляет блок разностью нового и старого значений, поэтому стоит $O(1)$. Это свойство именно суммы: для минимума удаление единственного минимального значения может потребовать просмотра всего блока.

5.2. Реализация суммы на C++17

Полный код находится в `code/05-decomposition.hpp`. Конструктор получает размер блока явно: это параметр модели нагрузки, а не обязательная скрытая константа. Массив может быть пустым. Все значения, разности и суммы должны помещаться в `long long`; длина ограничена миллионом элементов.

(c) Aleksey Tolstikov

```
class BlockSum {
    int n, width;
    std::vector<long long> a, total;
public:
    BlockSum(std::vector<long long> values, int b)
        : n(int(values.size())), width(b), a(std::move(values)) {
        assert(0 <= n && n <= 1000000 && width > 0);
        total.assign(n / width + (n % width != 0), 0);
        for (int i=0; i<n; ++i) total[i/width] += a[i];
    }
    void set(int i, long long x) {
        assert(0 <= i && i < n);
        total[i/width] += x-a[i];
        a[i] = x;
    }
    long long sum(int l, int r) const {
        assert(0 <= l && l <= r && r <= n);
        long long answer=0;
        while (l<r && l%width) answer += a[l++];
        while (width <= r-l) {
            answer += total[l/width]; l += width;
        }
        while (l<r) answer += a[l++];
        return answer;
    }
};
```

Первый цикл доводит левую границу до границы блока либо до конца запроса. Второй цикл берёт только полные блоки: условие `width <= r-l` не допускает выхода за правую границу. Третий дочитывает остаток. Все три цикла последовательно увеличивают одну границу, так что каждый элемент относится ровно к одной части разложения.

При присваивании к сумме блока прибавляется $x - a[i]$. Разность также должна быть представима в типе: одной гарантии, что исходные значения помещаются в `long long`, недостаточно. Здесь входные ограничения должны ограничивать промежуточную арифметику, как и в дереве отрезков.

Блоки не улучшают асимптотику суммы относительно дерева, но дают короткую реализацию и основу для менее удобных сводок. Выбор структуры нельзя делать только по длине кода: при большом количестве запросов суммы логарифмический алгоритм обычно предпочтительнее.

5.3. Откуда берётся квадратный корень

Для одного запроса оценка $B + n/B$ минимальна по порядку при $B \approx \sqrt{n}$. Это следует из равенства двух слагаемых или из неравенства $B + n/B \geq 2\sqrt{n}$. Не требуется, чтобы n было точным квадратом: округление до ближайшего положительного целого меняет лишь константу.

Но корень возникает из конкретной функции затрат. Пусть сводка требует $O(B)$ при обновлении и $O(1)$ при чтении полного блока. Для u обновлений и q запросов получаем

$$T(B) = O(n + (u + q)B + qn/B).$$

Баланс даёт $B \approx \sqrt{qn/(u+q)}$, с ограничением $1 \leq B \leq \max(1, n)$. Если обновлений гораздо больше запросов, полезны меньшие блоки. Для суммы обновление постоянно и этого дополнительного слагаемого uB нет. Если $q = 0$, оптимизация запросов вообще не определяет выбор параметра.

Реальные коэффициенты тоже различаются. Прямой просмотр последовательной памяти дешевле переходов по указателям; двоичный поиск внутри блока добавляет логарифм; копирование больших объектов дороже копирования целых. Сначала выпишите число действий, затем измеряйте несколько размеров блока на характерной нагрузке. Один удачный замер не доказывает универсальность $B = 700$.

Проверка понимания. Для $n = 10^6$, $q = 10^4$, $u = 10^6$ сравните модель с $B = 1000$ и с $B = 100$. Не требуется точный прогноз секунд: объясните, какое слагаемое уменьшилось и какое выросло. Затем повторите расчёт для суммы с постоянным обновлением.

5.4. Сортировка внутри блока и изменение контракта

Теперь запрос спрашивает число элементов на $[l, r)$, меньших x. Кроме исходного порядка сохраним отсортированную копию каждого блока. На краях сравним отдельные элементы с x; в полном блоке позиция `lower_bound(x)` равна числу подходящих значений. Время запроса $O(B + (n/B)\log(B+1))$, память $O(n)$.

После изменения одного элемента можно заново отсортировать его блок за $O(B \log(B+1))$. Более аккуратная реализация удаляет одно вхождение старого значения и вставляет новое в отсортированный вектор. Поиски логарифмические, но сдвиги элементов линейны: итоговое обновление $O(B)$. При дубликатах удаляют ровно одно вхождение, а не все равные значения.

Пусть исходный блок $[7, 2, 7, 1]$, сортированная копия $[1, 2, 7, 7]$. После замены позиции 0 на 3 копия становится $[1, 2, 3, 7]$. Запрос «меньше 7» меняется с 2 на 3. Индексы запроса относятся к исходному порядку; отсортированная копия нужна только для сводки. Сортировка самого массива разрушила бы смысл $[l, r)$.

В лекции 25.10.2022 около 15–17 минут обсуждается близкий, но другой интерфейс: коллекция из отсортированных блоков с вставкой и удалением элементов. В ней размеры блоков меняются, а поиск по позиции требует учитывать длины предыдущих блоков. Не переносите формулу `i/B` в такую структуру: она работает только для фиксированных границ.

Разбор по мотивам видео. При вставке нового значения сдвигается хвост одного блока. Если блок слишком вырос, его делят; слишком маленький присоединяют к соседнему и при необходимости делят результат. Это локализует перенос элементов. В новой расшифровке 16:20–16:41 лектор приводит опасный сценарий: всё время добавлять значения по убыванию, каждый раз в начало одного блока. Без разделения он растёт и перестаёт быть маленьким. Числовая адаптация: вставки 9, 8, 7, 6, 5 в один исходно пустой блок. Ограничение размера должно сработать независимо от того, насколько удобны остальные блоки.

5.5. Прыжки: сводка как переход в следующий блок

Пусть из позиции i выполняется прыжок в $i + a[i]$, причём $a[i] > 0$. Запрос начинает движение в заданной позиции и просит последнюю посещённую позицию и число прыжков до выхода за массив. Разрешено менять одно значение $a[i]$. Это учебная модель темы «Лунки» из плана курса; здесь используется нулевая индексация и явно зафиксирован контракт.

Прямой запрос может посетить весь массив. Для каждой позиции сохраним `next` — первую позицию вне её блока либо n, `count` — число прыжков до неё, `last` — последнюю позицию перед этим выходом. Сводка останавливается на границе блока и не пытается предсказать всю дальнейшую траекторию.

Для $a = [1, 2, 1, 1, 2, 1, 1, 3]$ и $B = 3$ траектория из 0: $0 \to 1 \to 3 \to 4 \to 6 \to 7 \to 10$. Первый блок заменяет два прыжка $0 \to 1 \to 3$ одним переходом сводки; второй — два прыжка $3 \to 4 \to 6$; последний — два прыжка $6 \to 7 \to 10$. Ответ: последняя позиция 7, всего 6 прыжков.

Построим один блок справа налево. Если $j = i + a[i]$ уже вне блока, то `count[i]=1`, `last[i]=i`, а `next[i]` равен j или внешнему маркеру n. Если j внутри, его сводка уже готова, поскольку $j > i$: добавляем один к числу прыжков и наследуем выход и последнюю позицию.

```
class JumpBlocks {
    int n, width;
    std::vector<int> a, next, count, last;
    void rebuild(int block) {
        int l=block*width, r=std::min(n,l+width);
        for (int i=r-1; i>=l; --i) {
            long long j=1LL*i+a[i];
            if (j>=r) {
                next[i]=int(std::min<long long>(n,j));
                count[i]=1; last[i]=i;
            } else {
                next[i]=next[int(j)];
                count[i]=1+count[int(j)];
                last[i]=last[int(j)];
            }
        }
    }
```

Изменение позиции может повлиять на более ранние позиции того же блока, поэтому перестраиваем весь блок за $O(B)$. Другие блоки менять не нужно: их сводки заканчиваются при первом выходе из собственного блока и не зависят от дальнейшей траектории. Во время запроса актуальная сводка следующего блока будет прочитана отдельно.

```
public:
    JumpBlocks(std::vector<int> jumps, int b)
        : n(int(jumps.size())), width(b), a(std::move(jumps)),
          next(n), count(n), last(n) {
        assert(n<=1000000 && 1<=width && width<=1000000);
        for (int x:a) assert(1<=x && x<=1000000000);
        for (int l=0; l<n; l+=width) rebuild(l/width);
    }
    void set(int i,int x) {
        assert(0<=i && i<n && 1<=x && x<=1000000000);
        a[i]=x; rebuild(i/width);
    }
    std::pair<int,int> exit_from(int i) const {
        assert(0<=i && i<n);
        int steps=0, final=i;
        while (i<n) {
            steps+=count[i]; final=last[i]; i=next[i];
        }
        return {final,steps};
    }
};
```

Листинги продолжают один класс. Полная реализация проверяет положительность прыжков. При нулевых или отрицательных значениях возможны циклы, зависимости перестают идти справа налево, а запрос уже не обязан завершаться. Это изменение постановки, а не редкий случай для небольшой правки цикла.

Каждый переход сводки увеличивает номер блока. Поэтому запрос стоит $O(1 + n/B)$, обновление $O(B)$, построение $O(n)$. Для u обновлений и q запросов балансируем $uB + qn/B$. При $u = 0$

можно предварительно посчитать переход до выхода из всего массива; динамическая структура тогда избыточна.

5.6. Тяжёлые и лёгкие вершины

Корневая декомпозиция не обязательно делит массив на отрезки. В простом неориентированном графе с m рёбрами сумма степеней равна $2m$. Если тяжёлой назвать вершину степени не меньше B, тяжёлых вершин не больше $2m/B$. Лёгких может быть много, зато их локальные списки коротки.

Рассмотрим подсчёт треугольников. Зададим строгий порядок вершин по паре `(степень, номер)` и направим каждое ребро от меньшей пары к большей. Вершина малой степени имеет мало исходящих рёбер. У вершины большой степени исходящие соседи тоже имеют большую степень, а таких вершин мало. При пороге около $\sqrt{2m}$ исходящая степень любой вершины равна $O(\sqrt{m})$.

Для каждой вершины u пометим её исходящих соседей. Затем для каждого ребра $u \to v$ просмотрим исходящих соседей w вершины v; если w помечен, найден треугольник. Наконец снимем метки. В каждом треугольнике есть единственная наименьшая вершина u и единственная средняя v, поэтому он учитывается один раз.

На пометки тратится $O(m)$, на двойные проходы — $O(m\sqrt{m})$, память списков и меток — $O(n + m)$. Строгий порядок по номеру при равенстве степеней важен: без него ориентация не определена однозначно. Петли и кратные рёбра заранее исключены контрактом; иначе «число треугольников» требует другого определения.

Здесь порог служит доказательству оценки, а не обязательному массиву с пометкой heavy. Общая схема полезна шире: дорогое действие выполняется либо над маленьким объектом, либо над небольшим числом больших объектов. Это другой источник квадратного корня, хотя алгебра баланса похожа.

5.7. Перестройка, перенос фрагментов и пределы метода

В той же лекции 25.10.2022 примерно с 27-й минуты появляется перенос фрагмента последовательности через `rotate`. Для $[a, b, c, d, e, f, g]$ и границ $(1, 4, 6)$ перестановка $[1, 4)$ и $[4, 6)$ даёт $[a, e, f, b, c, d, g]$. Буквы и границы этого примера редакторские. Порядок значений здесь не связан с сортировкой.

Блочное представление разрезает блоки, содержащие три границы, переставляет целые блоки и восстанавливает размеры около разрезов. Но цена перестановки самих описателей зависит от контейнера. Вектор указателей сдвигает описатели линейно по числу блоков; связный список быстро перепривязывает известные границы, зато ещё надо найти их по номеру позиции. Дешёвая локальная операция не делает весь запрос константным.

Если после каждой операции оставлять всё больше маленьких блоков, число сводок перестаёт быть $O(n/B)$. Нужен инвариант минимального размера либо периодическая полная перестройка. Например, если одна операция создаёт лишь постоянное число дополнительных кусков, после B операций их станет на $O(B)$ больше. Перестройка за $O(n)$ раз в B операций добавляет $O(n/B)$ амортизированной стоимости; её надо включить в баланс.

В расширенной практике задача «Архив» требует именно управления порядком последовательности. Она не сводится к нашему классу `BlockSum`. Сравните блочный подход с неявным деревом поиска: в дереве позиция задаётся размерами поддеревьев, а разрез и склейка действуют на всю последовательность. Сначала сравнивают интерфейсы и гарантии, затем выбирают структуру.

5.8. Другой контракт: прибавление диапазону и чтение точки

Сменим роли дорогого и дешёвого действия: теперь запрос читает один элемент, а обновление прибавляет число ко всему диапазону. Храним базовые значения элементов и одну отложенную прибавку на блок. Инвариант: логическое значение позиции i равно $base[i] + lazy[i/B]$.

При полном покрытии блока увеличиваем только его метку. При частичном покрытии увеличиваем базовые значения затронутых элементов, не трогая метку. В отличие от дерева с присваиваниями и прибавлениями, здесь все отложенные действия — сложения; они коммутируют. Поэтому можно изменить базовую часть и оставить прежнюю общую прибавку: итог будет тем же.

Для $[2, 7, 1, 4, 3, 6]$ и $B = 3$ выполним `add(0,3,5)`. Базовый массив не изменится, метки будут $[5, 0]$, а логический массив станет $[7, 12, 6, 4, 3, 6]$. Затем `add(1,5,2)` изменит базы позиций 1–4: получим базы $[2, 9, 3, 6, 5, 6]$ с теми же метками. Логический результат — $[7, 14, 8, 6, 5, 6]$.

```
class BlockAddPoint {
    int n, width;
    std::vector<long long> base, lazy;
public:
    BlockAddPoint(std::vector<long long> a,int b)
        : n(int(a.size())), width(b), base(std::move(a)) {
        assert(n<=1000000 && width>0);
        lazy.assign(n/width+(n%width!=0),0);
    }
    void add(int l,int r,long long x) {
        assert(0<=l && l<=r && r<=n);
        while(l<r && l%width) base[l++]+=x;
        while(width<=r-l) {lazy[l/width]+=x; l+=width;}
        while(l<r) base[l++]+=x;
    }
    long long get(int i) const {
        assert(0<=i && i<n); return base[i]+lazy[i/width];
    }
    void set(int i,long long x) {
        assert(0<=i && i<n); base[i]=x-lazy[i/width];
    }
};
```

Точечное присваивание также можно сделать без проталкивания: новое базовое значение равно $x - lazy[block]$. Например, `set(1,0)` в приведённом состоянии записывает в базу -5; чтение возвращает $-5 + 5 = 0$. Записав просто 0, мы получили бы ошибочный ответ 5. Отрицательная база допустима, даже если логические значения задачи неотрицательны.

Прибавление стоит $O(B + n/B)$, чтение и присваивание точки — $O(1)$; построение и память линейны. Все суммы и разности должны помещаться в `long long`. Структура не хранит суммы блоков и потому не даёт быстрых запросов суммы диапазона. Чтобы добавить такой запрос, надо поддерживать ещё один инвариант, как в § 3.

Этот пример объясняет, почему нельзя выбирать алгоритм по словам «диапазонные операции». Прибавление диапазону и чтение точки, присваивание диапазону и сумма диапазона, точечное изменение и статистика диапазона имеют разные удобные представления. Переносите прежде всего контракт, а не знакомый фрагмент кода.

(c) Aleksey Tolstikov

5.9. Углубление: декомпозиция порядка запросов

Иногда изменять данные вообще не нужно, а все запросы известны заранее. Пусть требуется число различных значений на каждом $[l, r)$ статического массива. Если умеем добавлять или удалять один элемент текущего окна, то можно не строить ответ заново, а перемещать две его границы. Частоты значений и число ненулевых частот позволяют менять ответ за $O(1)$ после сжатия координат.

Алгоритм Мо сортирует запросы сначала по номеру блока левой границы, затем по правой границе. В соседних блоках направление сортировки правой границы можно чередовать, чтобы уменьшить возвраты. Отвечаем в новом порядке, а результаты записываем под исходными номерами. В отличие от online-структуры, этот метод использует знание всего набора запросов.

Внутри одной группы левые границы лежат в блоке длины B, поэтому их перемещения дают $O(qB)$ за все запросы. Правая граница в каждой группе идёт монотонно по диапазону длины не больше n; групп $O(n/B)$, значит её перемещения стоят $O(n^2/B)$. Переходы между группами не меняют этот порядок. Вместе с сортировкой получаем $O(q \log(q+1) + qB + n^2/B + n)$.

При q порядка n подходит $B \approx \sqrt{n}$. При другом числе запросов баланс даёт $B \approx n/\sqrt{q}$, с отсечением до диапазона $[1, \max(1, n)]$. Если запросов нет, выполнять сортировку и перемещения незачем. Выбранный порядок не уменьшает цену добавления элемента: если она не постоянна, её надо умножить на число перемещений.

Для массива $[1, 2, 1, 3, 2]$ окно $[0, 3)$ содержит два различных значения. При переходе к $[1, 5)$ удаляется одна единица, но другая остаётся; затем добавляются 3 и 2, итог — три значения. Уменьшать число различных при каждом удалении неверно: оно меняется только при переходе частоты через ноль. Это минимальный контрпример для проверки операций окна.

Базовый алгоритм Мо предполагает статичный массив. Если между запросами есть обновления, одних двух координат окна недостаточно; добавляется измерение времени и другой анализ. Для суммы с точечными обновлениями дерево из § 2 обычно проще, а Мо полезен для статистики с дешёвым локальным изменением и неудобным объединением больших сводок.

5.10. Проверка и самостоятельная работа

Для суммы сравнивайте все ответы с прямым сложением. Включите размеры 0 и 1, длины $B-1$, B, $B+1$, запрос внутри одного блока, пустой запрос и неполный последний блок. Для прыжков эталоном служит буквальное движение по массиву; положительность значений гарантирует не более n шагов. Особенно полезно менять прыжок непосредственно перед границей блока.

Практика. Реализуйте подсчёт значений меньше порога с точечными изменениями. Затем решите модель прыжков и сравните её с наивным симулятором. Для углубления докажите оценки выбранного варианта переноса фрагмента: явно назовите контейнер блоков и стоимость поиска границ. Следующий параграф делит уже не данные на блоки, а пространство перебора на две половины.

5.11. Практика к главе

В расширенной подборке приложения A: №4473 «Лунки» (основная); №2741 «Архив» (углубление). Полные условия доступны по ссылкам в приложении. Выберите основные задачи, затем переходите к углублению; решать весь список за одну неделю не требуется.

5.12. Вопросы для самопроверки

1. Почему запрос внутри одного блока нельзя разложить на два независимо просматриваемых края?
2. Для какой сводки точечное присваивание постоянно, а для какой требует перестройки?
3. Выведите размер блока при разных количествах запросов и обновлений.
4. Почему сортированная копия блока не заменяет исходный порядок?
5. Что именно означают `next`, `count` и `last`?
6. Почему изменение прыжка не требует перестраивать предыдущие блоки?
7. Как нарушится доказательство при разрешении прыжка нулевой длины?
8. Докажите оценку числа тяжёлых вершин.
9. Почему ориентированный треугольник учитывается ровно один раз?
10. Как периодическая перестройка удерживает число маленьких кусков под контролем?

(c) Aleksey Tolstikov